Technical Stack: Decision-Stable Causal Attribution

Complete technology stack for building the project, with the Causal Constellation front end

Languages: Python (backend), JavaScript (frontend), HTML and CSS
Companion document: Decision-Stable Causal Attribution: Complete Technical Document

1. Architecture Decision

Build the system **precompute-first**, with an optional live backend. Python computes everything heavy (simulation, causal estimation, sensitivity bounds across the Gamma grid, experiment values) and writes JSON. The JavaScript front end only reads that JSON, so the demo works offline and cannot break from a server problem. A live backend is needed only for the "run experiment" action and for regenerating scenarios.

```
Python pipeline  ->  Parquet (journeys)  ->  results JSON (versioned schema)
                                                      |
                       static D3.js app  <------------+
                                                      |
                  optional FastAPI  <-- live experiment / scenario calls
```

2. Backend (Python)

Layer	Tool	Used for
Language	Python 3.11 or 3.12	Everything backend
Numerics and data	NumPy, pandas, SciPy	Simulator, features, bounds
Storage	Parquet via PyArrow, DuckDB for quick queries	Journey store (C2)
ML	scikit-learn, LightGBM	Cross-fitted propensity and outcome models (C4)
Statistics	statsmodels	Logistic baseline, standard errors, diagnostics
Optimization	CVXPY	Budget allocation and minimax regret (C7)

Layer	Tool	Used for
Causal cross-check	EconML or DoWhy (optional)	Sanity-check the custom AIPW against a library, not as the core
Deep learning (stretch only)	PyTorch	GRU propensity model for the time-varying estimator
Custom code	Plain NumPy	Last-click, Markov removal effect, sampled Shapley, sensitivity bounds, VOI Monte Carlo
Parallelism	joblib	200+ simulation replicates per scenario
Config	YAML with Pydantic models	Scenario and method configs, schema validation of the output JSON
Experiment tracking	Plain results folders with seeds and config hashes	Reproducibility
API (optional)	FastAPI with Uvicorn	Live "run experiment" and scenario endpoints
Testing and quality	pytest, ruff	Simulator acceptance tests, estimator unit tests
Environment	uv or Poetry with a pinned lockfile	Reproducible installs
Exploration	Jupyter	Prototyping only, not the final pipeline
Report figures	Matplotlib	Static plots for the written report

Note on the sensitivity bounds: write these yourself in NumPy or CVXPY from the published formulas, because you need to control and test them. Choose the exact bound formulation during the literature review.

3. Frontend (JavaScript, HTML, CSS)

Layer	Tool	Used for
Language	Modern JavaScript (ES modules)	App logic

Layer	Tool	Used for
Visualization	D3.js v7 (d3-force, d3-scale, d3-shape, d3-transition, d3-dispatch)	Constellation, stability map, all custom views
Quick charts	Observable Plot (optional)	Benchmark dashboard (V7), to save time
Rendering	SVG for stars, halos and outlines; SVG filters for glow; Canvas fallback if arcs get heavy	Constellation (V1)
Markup and styling	HTML5, CSS3 with CSS variables, grid and flexbox	Layout, dark theme
Build	Vite	Dev server and bundling
State	A small store built on d3-dispatch	Keeping linked views in sync (Gamma, selected star, method, scenario)
Testing (optional)	Vitest for logic, Playwright for a smoke test	Catching broken views before the demo

Why no React: the views are tightly linked D3 visuals, and D3 manipulating the DOM directly is simpler than fighting a framework. Use React or Svelte only if you already know it well, and in that case keep D3 for scales and layout while the framework renders the elements.

4. Component to Stack Map

Component	Stack
C1 Simulator	NumPy, pandas, joblib, pytest
C2 Data layer	Parquet, DuckDB, Pydantic schemas
C3 Baselines	NumPy, pandas, scikit-learn
C4 Causal estimator	LightGBM, scikit-learn cross-fitting, NumPy, optional PyTorch
C5 Overlap diagnostics	NumPy, statsmodels, Matplotlib for checks
C6 Sensitivity engine	NumPy, CVXPY, SciPy
C7 Decision layer	CVXPY, NumPy

Component	Stack
C8 Experiment selector	NumPy Monte Carlo, joblib
C9 Evaluation harness	joblib, pandas, Matplotlib
C10 Visual interface	D3.js, SVG, CSS, Vite
C11 Real-data module	pandas, DuckDB, same estimator code

5. Data Contract

- One versioned JSON schema per output, validated in Python with Pydantic before writing.
- Precompute the Gamma grid (for example 1.0 to 3.0 in steps of 0.1) for every scenario and method, so the slider is instant and needs no server.
- Keep the files small, a few MB in total, so the app loads fast.

6. Deployment and Delivery

- **Demo mode:** a static build (`vite build`) hosted on GitHub Pages or Netlify, or opened locally.
- **Live mode:** FastAPI on a laptop for the "run experiment" action.
- **Backup:** a screen recording of the full demo in case of a live failure.
- **Reproducibility:** a Makefile or single script with targets such as `make sim`, `make evaluate`, `make export`, `make web`.

7. Hardware

A normal laptop is enough. Keep the simulator at roughly 50k to 200k journeys for the full replicate grid and run replicates in parallel with joblib. No GPU is needed unless the GRU stretch goal is attempted.

8. Repository Layout

```
project/
  configs/  sim/  methods/  decision/  experiments/  eval/
  api/        FastAPI app (optional)
  viz/        Vite + D3 front end, results JSON
  tests/  report/
```

9. Working With Coding Agents

1. Give the agent the JSON schemas and the component spec first.
2. Build the Python and JavaScript sides separately against the schema.
3. Write the simulator's acceptance tests before the estimator.
4. Treat statistical results with suspicion: bugs produce plausible but wrong numbers, so these tests matter more than visual polish.
5. Commit often so a bad agent change can be undone.